AIAC-20.3

Name: Siri Vempati

Ht.no:2303A51309

Batch.no:05

Task-1

Prompt

Generate a Python user registration program that validates password strength. The program should:

- Enforce a minimum password length of 8 characters
- Use regular expressions to ensure the password contains uppercase, lowercase, digits, and special characters
- Reject commonly used weak passwords
- Display appropriate error messages until a strong password is entered

Code:

```python
import re
COMMON_PASSWORDS = {
    "password", "123456", "12345678", "qwerty",
    "abc123", "password123", "111111", "123123"
}
def is_strong_password(password):
    # Check minimum length
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."

    # Check common passwords
    if password.lower() in COMMON_PASSWORDS:
        return False, "Password is too common."

    # Regex for strong password
    pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).+$'

    if not re.match(pattern, password):
        return False, "Password must include uppercase, lowercase, number, and special character."

    return True, "Strong password!"
def register_user():
    print("=== User Registration ===")

    username = input("Enter username: ")

    while True:
        password = input("Enter password: ")
        valid, message = is_strong_password(password)
        if valid:
            print("✅ Registration successful!")
            break
        else:
            print("❌", message)
# Run program
register_user()
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:/Users/vempa/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/vempa/OneDrive
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/Ass-19.2/ass-20.3.py"
=== User Registration ===
Enter username: siri
Enter password: 12345
❌ Password must be at least 8 characters long.
Enter password: Vempati@123
✅ Registration successful!
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING>
```

Task-02

Prompt

Generate a secure Flask-based file upload system that:

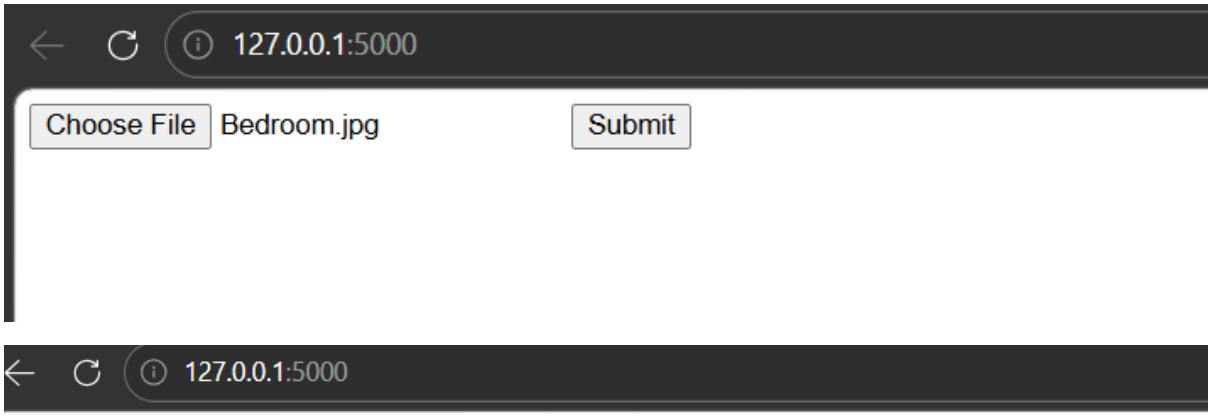
- Validates file type (only allows safe extensions like png, jpg, pdf)
- Restricts file size (e.g., max 2MB)
- Stores files safely using secure filenames
- Prevents malicious file uploads

Code:

```python
from flask import Flask, request, redirect
import os
from werkzeug.utils import secure_filename
app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'pdf'}
MAX_FILE_SIZE = 2 * 1024 * 1024  # 2MB
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
def allowed_file(filename):
    return '.' in filename and \
           filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            return "No file part"
        file = request.files['file']
        if file.filename == '':
            return "No selected file"
        # Validate file type
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return "File uploaded successfully ✅"
        else:
            return "Invalid file type ❌"
    return '''
    <form method="post" enctype="multipart/form-data">
        <input type="file" name="file">
        <input type="submit">
    </form>
    '''
if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/app.py"
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:/Users/vempa/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/vem
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/app.py"
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 112-702-995
```



Explanation

- Validates **file type** → only allows safe formats (png, jpg, pdf)
- Limits **file size (2MB)** → prevents server overload
- Uses secure_filename() → avoids malicious filenames
- Stores files in a **safe folder** → prevents path attacks

Result: Blocks harmful uploads and keeps the server secure

Task-03

Prompt

Generate a secure Flask API endpoint with proper security practices. The API should:

- Use token-based authentication (JWT or API key)
- Validate user input
- Restrict HTTP methods properly
- Implement rate limiting to prevent abuse
- Return secure and structured responses

Code:

```python
from flask import Flask, request, jsonify
from functools import wraps
import time
app = Flask(__name__)
API_KEY = "mysecureapikey123"
UPLOAD_LIMIT = 5  # requests per minute
request_log = {}
def rate_limit(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        ip = request.remote_addr
        current_time = time.time()
        if ip not in request_log:
            request_log[ip] = []
        request_log[ip] = [t for t in request_log[ip] if current_time - t < 60]
        if len(request_log[ip]) >= UPLOAD_LIMIT:
            return jsonify({"error": "Too many requests"}), 429
        request_log[ip].append(current_time)
        return func(*args, **kwargs)
    return wrapper
def require_api_key(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        key = request.headers.get("x-api-key")
        if key != API_KEY:
            return jsonify({"error": "Unauthorized"}), 401
        return func(*args, **kwargs)
    return wrapper
@app.route('/')
def home():
    return "API is running successfully 🚀"
@app.route('/api/data', methods=['POST'])
@require_api_key
@rate_limit
def secure_api():
    data = request.get_json()
    if not data or "name" not in data:
```

```python
def secure_api():
        return jsonify({"error": "Invalid input"}), 400
    name = data["name"]
    if not isinstance(name, str) or len(name) < 2:
        return jsonify({"error": "Invalid name"}), 400
    return jsonify({
        "message": f"Hello, {name}! API is secure."
    })
if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/app(20.3)(3).py"
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:/Users/vempa/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/vempa/OneDrive/Pi
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/app(20.3)(3).py"
 * Serving Flask app 'app(20.3)(3)'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 112-702-995
127.0.0.1 - - [01/Apr/2026 15:17:16] "GET / HTTP/1.1" 200 -
```

404 Not Found | 127.0.0.1:5000

127.0.0.1:5000

API is running successfully 🚀

Explanation

- **Authentication**: Uses an API key (x-api-key) to allow only authorized users
- **Rate Limiting**: Limits requests (5 per minute per IP) to prevent abuse and attacks
- **Input Validation**: Ensures the request contains valid JSON and a proper name field
- **Secure Endpoint**: Uses only the POST method for controlled access
- **Home Route**: Adds / route to avoid 404 errors

Result: The API is secured against unauthorized access, invalid data, and excessive requests.

Task-04

Prompt:

Generate a secure HTML feedback form with JavaScript that:

- Accepts user input (name and feedback)
- Displays the output safely on the page
- Prevents XSS attacks by escaping HTML entities
- Avoids using unsafe methods like innerHTML
- Implements basic Content Security Policy (CSP)

Code:

app.py app(20.3)(3).py app(20.3)(4).html

app(20.3)(4).html > html > body > script

```html
<html>
<head>
    <title>Secure Feedback Form</title>
    <meta http-equiv="Content-Security-Policy"
          content="default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';">
</head>
<body>
<h2>Feedback Form</h2>
<form id="feedbackForm">
    <input type="text" id="name" placeholder="Enter your name" required><br><br>
    <textarea id="feedback" placeholder="Enter feedback" required></textarea><br><br>
    <button type="submit">Submit</button>
</form>
<h3>Output:</h3>
<p id="output"></p>
<script>
document.getElementById("feedbackForm").addEventListener("submit", function(event) {
    event.preventDefault();

    let name = document.getElementById("name").value;
    let feedback = document.getElementById("feedback").value;

    // Escape HTML entities
    function escapeHTML(str) {
        return str.replace(/&/g, "&amp;")
                  .replace(/</g, "&lt;")
                  .replace(/>/g, "&gt;")
                  .replace(/"/g, "&quot;")
                  .replace(/'/g, "&#039;");
    }
    let safeName = escapeHTML(name);
    let safeFeedback = escapeHTML(feedback);
    document.getElementById("output").textContent =
        "Name: " + safeName + " | Feedback: " + safeFeedback;
});
</script>
</body>
```

Ln 17, Col 9 Spaces: 4 UTF-8 CRLF {} HTML

Output:

Explanation

- Prevents XSS by **escaping HTML characters** (<, >, &, etc.)
- Uses **textContent instead of innerHTML** to safely display input
- Adds **Content Security Policy (CSP)** to block unsafe scripts

Result: User input is displayed safely without executing malicious code.

Task-05

Prompt

Generate a secure Python script that fetches user details from a SQLite database. The script should:

- Avoid SQL injection vulnerabilities
- Use parameterized queries (prepared statements)
- Take user input safely
- Return matching user records securely

Code:

app.py | app(20.3)(3).py | app(20.3)(4).html | app(20.3)(5).py | Task-05(3.3).py

app(20.3)(5).py > fetch_user

```python
import sqlite3
def setup_database():
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT NOT NULL,
        email TEXT NOT NULL
    )
    """)
    cursor.execute("SELECT COUNT(*) FROM users")
    if cursor.fetchone()[0] == 0:
        cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("admin", "admin@example.com"))
        cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("siri", "siri@example.com"))
        conn.commit()
    conn.close()
def fetch_user():
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    username = input("Enter username: ")
    query = "SELECT id, username, email FROM users WHERE username = ?"
    cursor.execute(query, (username,))
    result = cursor.fetchone()
    if result:
        print("User Found:")
        print("ID:", result[0])
        print("Username:", result[1])
        print("Email:", result[2])
    else:
        print("No user found")
    conn.close()
setup_database()
fetch_user()
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
p/HCL LAB/AI ASSISTANT CODING/app(20.3)(5)"
No user found
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:/Users/vempa/AppData/Local/Programs/Python/Python311
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/app(20.3)(5).py"
Enter username: admin
User Found:
ID: 1
Username: admin
Email: admin@example.com
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING>
```

Explanation

Uses **parameterized query (?)** instead of string concatenation ,Prevents SQL injection attacks (e.g., ' OR '1'='1) ,Treats user input as **data, not executable SQL** ,Ensures safe database access.